



Redundancy & Failover in AI Infrastructure

Building resilient AI systems that can withstand hardware, network, and regional outages through strategic redundancy and automated failover mechanisms.

Why It Matters

AI APIs must remain **always-on** for critical applications:

- Fraud detection systems
- Healthcare monitoring
- Autonomous driving

Downtime leads to:

- Lost revenue
- SLA penalties
- Erosion of customer trust

Capstone Goal: Deliver $\geq 99.9\%$ uptime



Failures are *inevitable* — the key is designing for **graceful recovery**

Types of Redundancy

1

Horizontal

Multiple replicas of the same service running in parallel, allowing for load distribution and failover capability

2

Vertical

Over-provisioning capacity per node (less common in AI infrastructure but useful for handling unexpected load spikes)

3

Geographic

Deploying across multiple availability zones and regions to protect against localized failures

4

Storage

Replicating datasets and model checkpoints across multiple storage systems and locations

5

Networking

Implementing multiple load balancer endpoints and DNS failover mechanisms

Redundancy at Each Layer

Compute

Multi-node, multi-AZ GPU pools to distribute processing load and provide failover capacity

Data

Object storage replication (S3 CRR, GCS dual-region) for data durability across regions

Network

Load balancer replicas and redundant network paths to prevent single points of failure

Control Plane

Kubernetes multi-master setup with Etcd quorum for resilient cluster management

Application

Replicated pods with PodDisruptionBudget (PDB) and canary deployment options

Failover Mechanisms

Active-Active

All replicas simultaneously serve traffic, distributed by load balancers

Active-Passive

Standby infrastructure activated only during failover events

DNS Failover

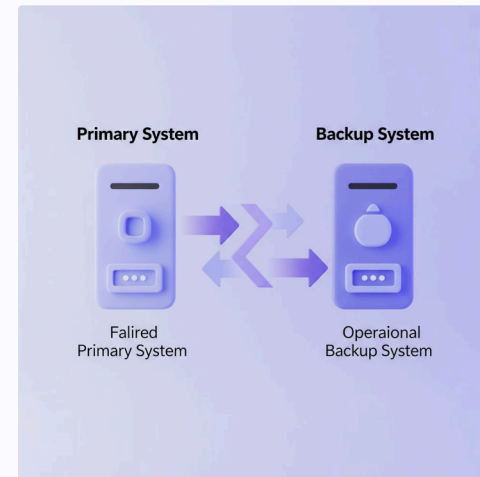
Switch traffic between regions via Route53/Cloud DNS when health checks fail

Cluster Failover

Promote standby cluster to primary when outage is detected

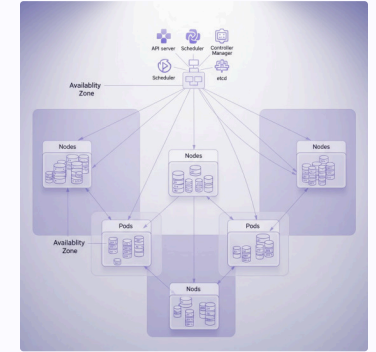
Database Failover

Automatic leader election or promotion of read-replicas during primary failure



Kubernetes HA Patterns

- Deployments with **replicas ≥ 3** for critical services
- PodAntiAffinity to spread workloads across nodes/AZs
- **PodDisruptionBudget (PDB)** to prevent all replicas from being drained
- Cluster autoscaler with **multi-zone node pools**



```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: infer-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: infer
```

This PDB ensures at least 2 pods remain available during voluntary disruptions

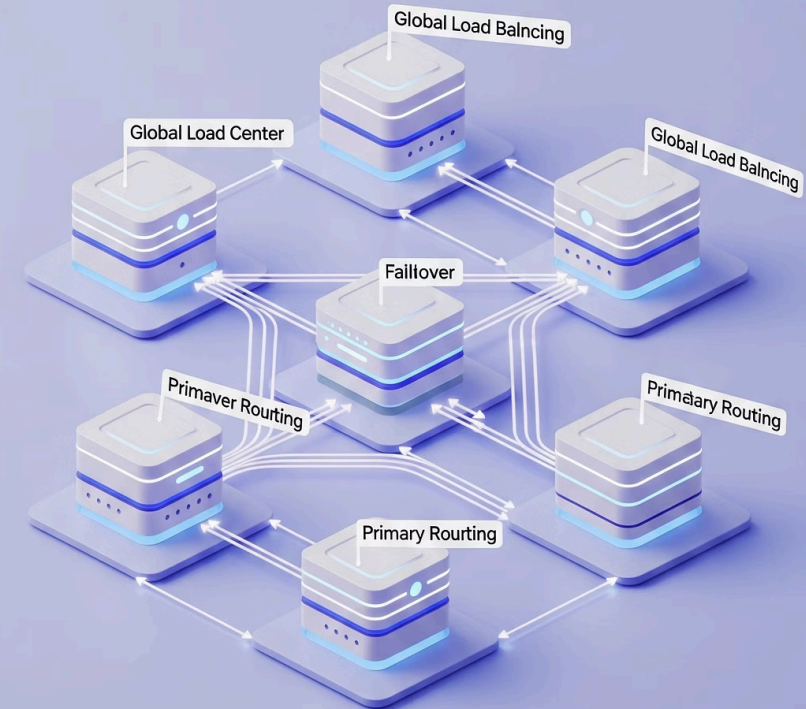
Load Balancing & DNS Failover

Load Balancing

- **L4/L7 Load Balancers** distribute traffic across healthy replicas
- Health checks continuously verify endpoint availability
- Global Anycast IPs enable low-latency routing worldwide

DNS Failover

- Health checks trigger automatic traffic rerouting
- Example: AWS Route53 + CloudFront redirect traffic to healthy region
- TTL settings control propagation speed during failover



Storage Redundancy



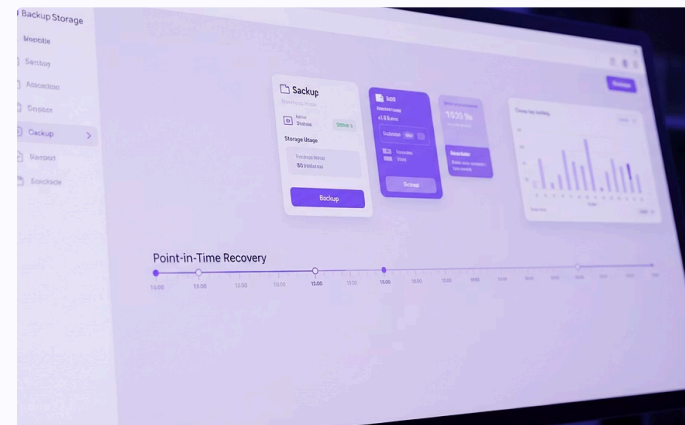
Cross-Region Replication

Enable automatic cross-zone replication on object storage (S3 CRR, GCS dual-region) to protect model weights and datasets



Multi-Region Databases

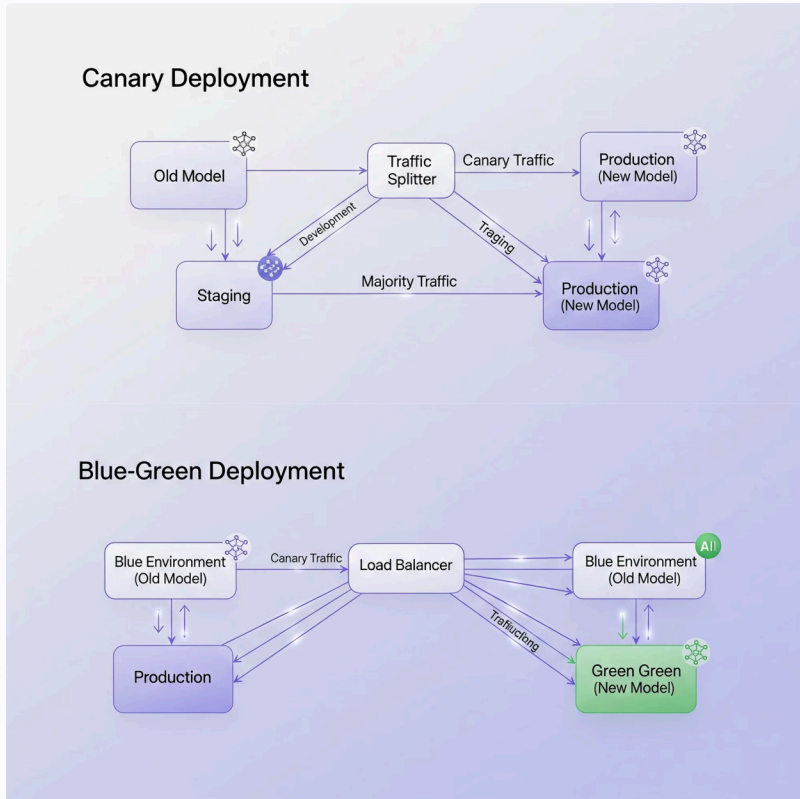
Implement globally distributed databases like Aurora Global Database, CosmosDB, or Spanner for consistent data access



Backup Strategy

Follow the 3-2-1 rule: **3** copies, **2** different storage types, **1** offsite backup

Failover for Model Serving



Canary & Blue/Green Deployments

Enable instant rollback capabilities when new model versions fail

Shadow Mode

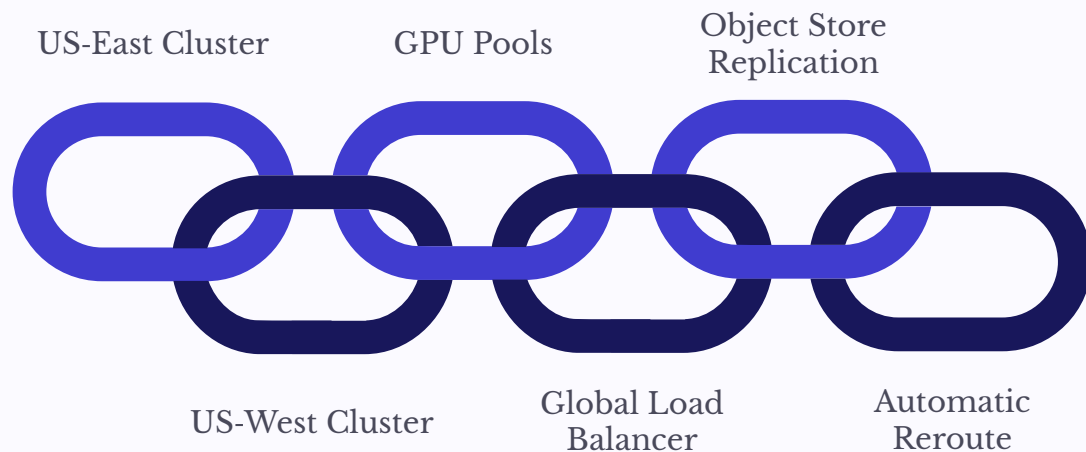
Run standby model versions that receive traffic but don't return responses

Service Mesh

Use Istio/Linkerd for fine-grained traffic control:

v1 = 90%, v2 = 10% → failover to stable version if v2 fails

Multi-Region Example



This architecture provides resilience against:

Regional Outages

Complete failure of a datacenter or region

Network Partitions

Connectivity issues between regions

Resource Exhaustion

GPU capacity limits in a single region

Testing Failover (Drills)

Chaos Engineering Approach

- Deliberately kill pods/nodes using Chaos Mesh or Kube-monkey
- Block network traffic to a region for 15 minutes
- Simulate GPU failures or performance degradation

Practice **GameDays** regularly with cross-functional teams

Verification Checklist

- Load balancers correctly reroute traffic to healthy endpoints
- Client applications retry successfully with minimal errors
- Monitoring dashboards clearly show the failover event
- Recovery happens within SLA time constraints



Cost vs Reliability Trade-off



Multi-Zone

+15-20% cost

High ROI with protection against zone failures

Recommended minimum for all production AI systems



Multi-Region Active-Active

+50-100% cost

Maximum uptime with simultaneous operation in multiple regions

Ideal for mission-critical AI applications requiring 99.99%+ uptime



Active-Passive

Lower ongoing costs, higher RTO

Standby infrastructure spun up only during failover events

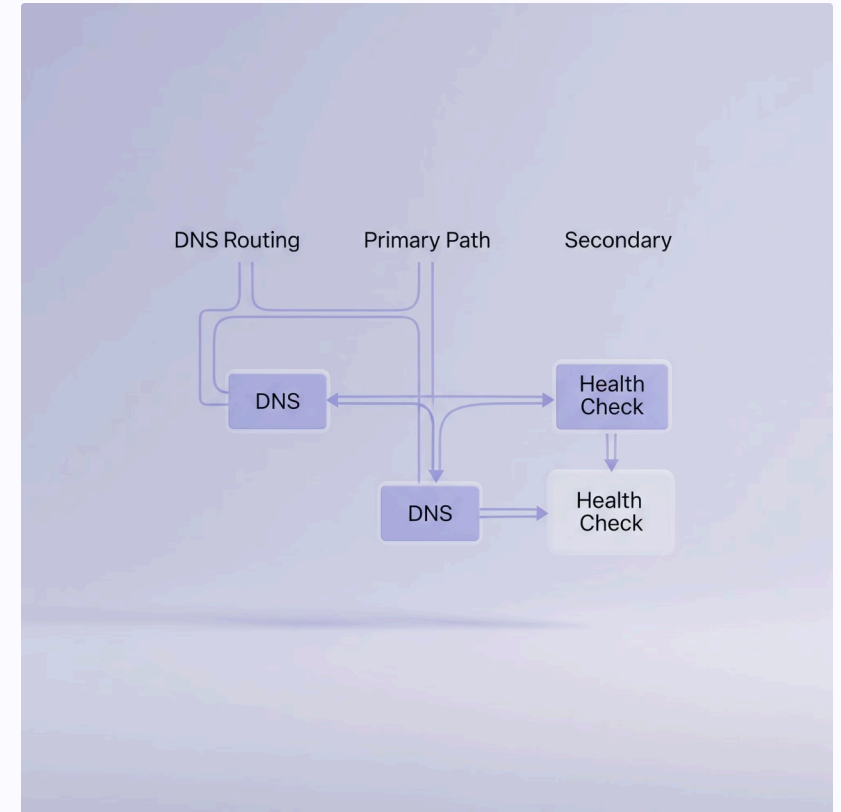
Balance point between cost and reliability

Choose your redundancy strategy based on your specific [SLA requirements](#) and budget constraints

Example Config: DNS Failover

```
# Route53 health check
HealthCheckConfig:
  Type: HTTP
  ResourcePath: /healthz
  FailureThreshold: 3

# Failover routing policy
RecordSets:
- Name: api.capstone.com
  Type: A
  SetIdentifier: us-east
  Failover: PRIMARY
  HealthCheckId: [...]
- Name: api.capstone.com
  Type: A
  SetIdentifier: us-west
  Failover: SECONDARY
  HealthCheckId: [...]
```



This configuration automatically routes traffic to the secondary region when the primary health check fails

Observability for High Availability

Critical Metrics & Dashboards

- Uptime percentage by service and region
- Failover events with time-to-recovery
- Resource utilization across zones
- Error rates during normal and failover operations

Track RTO (Recovery Time Objective) & RPO (Recovery Point Objective) as metrics

Alert Configuration

Cluster quorum loss detected

Load balancer health check failures

Elevated error rates above threshold

Cross-region replication lag



Best Practices for Capstone



Redundant Services

Always maintain ≥ 3 replicas for each critical service



Geographic Distribution

Spread across ≥ 2 Availability Zones; consider ≥ 2 regions for critical applications



Automation

Fully automate failover processes — don't rely on manual operations



Regular Testing

Conduct frequent failover drills — trust but verify your redundancy measures



Summary

Key Concepts

Redundancy = Extra capacity to absorb failures

Failover = Automatic rerouting when systems fail

Redundancy Layers

- Compute & GPU resources
- Storage & data replication
- Network paths & load balancing
- Control plane & application layer

Implementation Strategy

- High availability costs extra — balance against SLA requirements
- Start with multi-AZ, then consider multi-region for critical services
- Test failover mechanisms like any other feature
- Conduct regular chaos engineering drills



Discussion

For your Capstone, would you choose multi-AZ active-active or multi-region active-passive?

How does your SLA and budget shape that choice?

Consider:

- What recovery time can your application tolerate?
- What's the cost impact of downtime for your service?
- How do your infrastructure costs compare to business impact?

